

Auditsysteem voor Eurovision contest



Adam Bouchikhi (r1048050)
Arsalan Bornha (r1035349)
Larissa Mbonazo Lusakuono (r1029256)
Mohamed Amin Ahkim (r1038196)
Saartje Cordier (r1037117)
Tweede Bachelor Toegepaste Informatica

Integration Project

Academiejaar 2025-2026

Inleiding

Het is nog nooit zo belangrijk geweest om grote hoeveelheden data te kunnen verwerken in een beperkte tijd. Een klassiek voorbeeld waarbij dit onmiddellijk duidelijk wordt is de Eurovision Song Contest. Hier wordt in korte tijd een enorme datastroom gegenereerd afkomstig uit vele landen.

Zonder een correcte en efficiënte verwerking is het onmogelijk om tot een betrouwbare ranking te komen. Maar zo'n evenementen genereren piekbelastingen die traditionele systemen gewoonweg niet aankunnen en dus zijn schaalbare, performante oplossingen noodzakelijk.

Wij gaan in dit project een proof of concept ontwikkelen voor een systeem dat deze stemdata via een gedistribueerde pipeline kan verwerken. Hiervoor onderzoeken en gebruiken we technieken zoals Apache Kafka, Apache Spark en Apache Cassandra.

Probleemstelling & Doelstelling

Probleem

De Eurovision Song Contest verwerkt jaarlijks een enorm grote hoeveelheid stemmen afkomstig uit verschillende landen. Deze stemmen worden in een korte tijd verzameld en moeten snel en correct verwerkt worden. Om zo tot een definitieve ranking te komen.

Het verwerken van deze grote datastroom is een uitdaging. Traditionele, niet-geschaalde oplossingen kunnen het lastig krijgen met de zware belasting, dit kan vertragingen en fouten in de verwerking tot gevolg hebben. Dit is dan ook de reden dat het noodzakelijk is om een oplossing te zoeken die schaalbaar, betrouwbaar en efficiënt is.

Boven op de verwerking moet het systeem ook controles kunnen uitvoeren om de juistheid van de gegevens te kunnen aantonen. Dit zorgt dan weer voor een hogere complexiteit. Een goed ontworpen, gedistribueerde aanpak is dus noodzakelijk.

Doel

Het doel van dit project is het ontwikkelen van een proof of concept (PoC) voor een schaalbaar systeem dat in staat is om de stemmen van de Eurovision Song Contest correct te verwerken en een juiste ranking op te stellen.

De focus van ons project ligt op de volgende punten:

- verwerking van de stemmen via een gedistribueerde architectuur
- berekenen van een concrete en consistente ranking
- uitvoeren van controles

Het is niet de bedoeling dan we een volledig afgewerkt product afleveren, maar wel dat we een prototype maken om aan te tonen dat het zou werken in een realistische omgeving.

Afbakening van het project

Aan het begin van het project werden er een aantal duidelijke afspraken gemaakt:

- stemmen zijn enkel in SMS-formaat
- 1 SMS = 1 stem, er wordt geen gebruik gemaakt van het puntensysteem
- het systeem berekent het aantal stemmen per land
- er wordt een ranking opgesteld
- het is een passief systeem dat als controle werkt langs het officiële

Aanpak & Planning

Wij kozen ervoor om in fasen te werken, maar op een iteratieve manier. Aangezien het doel was om een schaalbaar systeem te ontwikkelen voor het verwerken van stemmen, konden wij het project opdelen in een aantal logische stappen. Deze bouwden telkens op elkaar voort.

In de eerste fase focusten wij op analyse en verwerking. We onderzochten het probleem en de mogelijke technologieën en architecturen die we konden gebruiken om grote hoeveelheden data te verwerken. Vervolgens hebben we ons verdiept in de gekozen technologieën. Dit vormde de basis voor de verdere uitwerking van ons project.

Vervolgens heeft iedereen een PoC uitgewerkt in een lokale omgeving. Hier focusten we ons puur op de basis, namelijk het verwerken van de stemmen en het genereren van een correcte ranking. Dit gaf ons de kans om onze aanpak uit te testen zonder deze al te ver uit te werken.

In een volgende stap hebben we ervoor gekozen om het systeem uit te breiden en aan te passen naar een schaalbaardere en realistischere omgeving. Om de verwerking efficiënter en flexibeler te maken hebben we dan ook gebruik gemaakt van containerinstanties en cloudtechnologieën.

Ten slotte hebben we ons gefocust op het verfijnen en testen van het systeem. Hierbij focusten wij ons vooral op de correctheid, performantie en betrouwbaarheid.

De planning bestaat uit de volgende fasen:

- analyse en voorbereiding
- ontwikkeling van de PoC
- uitbreiding van de architectuur
- testen en evalueren

Requirements

Deze hebben we afgeleid uit de analyse en doelstellingen.

Functioneel

het systeem moet:

- F-R01 - stemdata kunnen opslaan
- F-R02 - inkomende stemmen kunnen verwerken
- F-R03 - een correcte ranking berekenen op basis van stemmen
- F-R04 - de resultaten van de stemming kunnen weergeven
- F-R05 - controles kunnen uitvoeren
- F-R06 - nieuwe stemmen blijven verwerken terwijl het systeem actief is
- F-R07 - een werkende pipeline hebben

Niet-functioneel

Het systeem moet:

- NF-R01 - schaalbaar zijn voor grote hoeveelheden data
- NF-R02 - performant zijn voor snelle berekeningen
- NF-R03 - betrouwbare en consistente resultaten opleveren
- NF-R04 - robuust zijn en blijven werken bij hoge belasting
- NF-R05 - flexibel zijn om makkelijk uit te kunnen breiden
- NF-R06 - beschikbaar blijven tijdens de hele stemperiode
- NF-R07 - efficiënt omgaan met resources

Architectuur & Systeemoverzicht

Overzicht pipeline

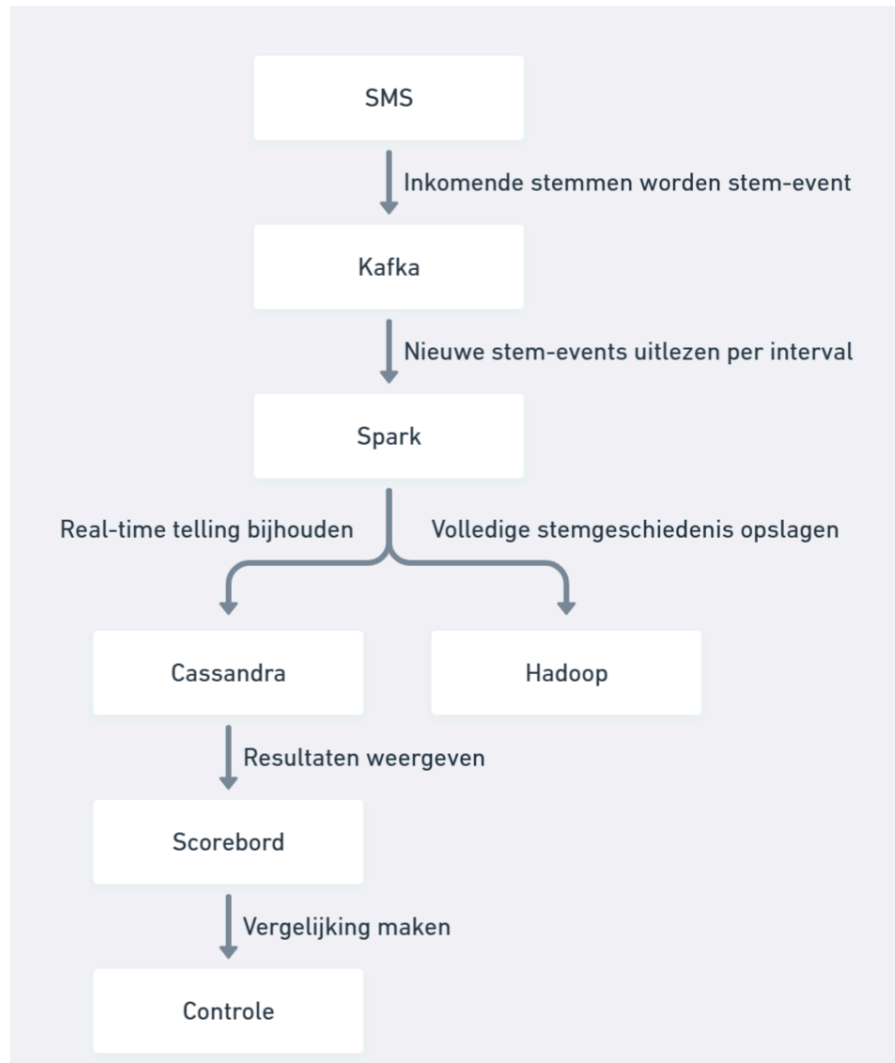
Het systeem dat we hebben ontwikkeld is gebouwd als een pipeline. Met andere woorden, de data die wordt ontvangen stroomt stap voor stap doorheen ons systeem en wordt bij elke stap een beetje meer verwerkt.

Het begint met de inkomende stemmen die worden omgezet naar een standaardformaat om de verwerking makkelijker te maken. Hierna worden ze doorgestuurd naar Kafka, een soort buffer die ervoor zorgt dat er geen stemmen verloren gaan.

Dan begint de verwerking van de data in real-time door Apache Spark. Het aantal stemmen wordt geteld per land en er wordt een ranking berekend. Deze verwerking is continu, het blijft doorgaan terwijl er nieuwe stemmen binnenkomen.

Na de verwerking worden de resultaten opgeslagen in Cassandra. In een uitgebreidere versie kan Hadoop gebruikt worden voor het archiveren van data.

Ten slotte worden de resultaten weergegeven in een ranking. Deze kan dan vergeleken worden met de officiële ranking.



Figuur 1: Overzicht van de data pipeline voor het verwerken van stemmen, van inkomende SMS tot opslag en controle

Datacenter & Cloud

Ook de infrastructuur van het systeem is belangrijk om even bij stil te staan. Het systeem draait namelijk niet op één enkele computer maar bestaat uit verschillende onderdelen bestaande uit virtuele machines en containers.

We maakten gebruik van een virtuele machine waarop verschillende tools zijn geïnstalleerd. In de VM maken we gebruik van containers om de verschillende componenten, zoals Kafka, Spark en Cassandra afzonderlijk te laten draaien.

Dan maakten we een meer uitgebreide opstelling waarbij het systeem kan worden verdeeld over meerdere datacenters en servers. Dit spreidt de verwerking waardoor het systeem veel beter bestand is tegen hoge belasting. Deze soort opstelling zorgt ervoor dat meerdere instanties van bijvoorbeeld Kafka of Spark tegelijkertijd kunnen draaien, wat de schaalbaarheid vergroot.

De architectuur kan ook in een cloudomgeving draaien, zoals via Google Cloud. Wij gebruiken hier verschillende virtuele machines die verschillende onderdelen van het systeem hosten. Dit zorgt voor een hogere flexibiliteit en schaalbaarheid.

Componenten

Het systeem bestaat uit verschillende belangrijke componenten die elk een specifieke rol hebben:

- Apache Kafka
Dit wordt gebruikt als een soort buffer die de inkomende datastroom bijhoudt. Wat ervoor zorgt dat er geen data verloren gaat of corrupt wordt. Het draagt bij aan een vlotte verwerking.
- Apache Spark
Hier gebeurt de verwerking, filtering en telling van de stemmen. Hier wordt de ranking bepaald.
- Apache Cassandra
Dit slaat de actuele data op. De telling wordt hierin bewaard. Dit omdat Cassandra geschikt is voor snelle verwerking
- Hadoop
Het wordt gebruikt als opslag voor historische data. Hadoop kan gebruikt worden voor de opslag van grotere hoeveelheden data, deze kunnen later geanalyseerd worden.
- Docker
Dankzij docker kunnen we alle verschillende onderdelen in containers draaien. Dit ondersteunt een consistente werking en zorgt ervoor dat alles makkelijk te implementeren is.

Data model

In dit project werken we met stemdata. Elke stem wordt opgeslagen als een aparte record in ons systeem. Het datamodel is simpel in elk record worden de volgende dingen bijgehouden:

- vote_id
Een unieke identifier per stem. Dit vermijdt duplicaten.
- country
Dit toont voor welk land gestemd werd. Dit wordt gebruikt om de stemmen te tellen en de ranking op te stellen.
- timestamp
Dit veld toont wanneer gestemd werd. Dat is belangrijk zijn voor real-time verwerking en latere analyse.
- voter_ip
Het IP-adres van de stemmer kan gebruikt worden voor controle en validatie van de stemmen.

Deze structuur maakt het mogelijk om later makkelijk beperkingen uit te voeren, wanneer de data verder door de pipeline gaan. Zo kunnen alle stemmen worden verwerkt en kan er een ranking worden berekend.

Technologische keuzes

Inleiding

Na ons onderzoek bleek al snel dat dit project schaalbaarheid en efficiëntie van groot belang waren. Het systeem moet grote hoeveelheden data in real-time kunnen verwerken. Dit is dan ook waarom we in de wereld van big data zijn gekropen en hier verschillende technologieën hebben onderzocht en gekozen om verder te gebruiken.

Deze gekozen technologieën vormen de basis van ons gehele project en zorgen ervoor dat de verwerking van de stemmen correct en performant kan verlopen.

Apache Kafka

Apache Kafka wordt gebruikt als datastreaming platform. Kafka zal de inkomende stemmen opvangen en tijdelijk bufferen.

Tijdens de stemming kunnen plots grote pieken ontstaan. Dankzij Kafka zal deze data niet verloren gaan en steeds aan een stabiel tempo kunnen blijven verwerkt worden. Het is een soort tussenlaag tussen de input en de verwerking van de data.

Apache Spark

Apache Spark wordt gebruikt voor dataverwerking. Het zal de stemmen in real-time analyseren en verwerken.

Spark zal de data uit Kafka uitlezen en hierop bewerkingen uitvoeren, zoals het tellen van de stemmen en het berekenen van de ranking. Spark heeft de mogelijkheid om data parallel te verwerken, dit maakt het ideaal om grote hoeveelheden data snel te verwerken.

Apache Cassandra

Apache Cassandra is de database waar de resultaten worden opgeslagen. Het is een NoSQL database dat is ontworpen om grote hoeveelheden data met hoge snelheden te verwerken.

Wij gebruiken Cassandra om de verwerkte stemresultaten op te slaan.

Apache Hadoop

Apache Hadoop wordt gebruikt om historische gegevens op te slaan. In tegenstelling tot Cassandra, kan Hadoop grote datasets op lange termijn bewaren.

Hadoop maakt het mogelijk om achteraf analyses of controles uit te voeren op de volledige dataset.

Docker

Docker wordt gebruikt om de verschillende onderdelen van het systeem in containers te draaien. Hierdoor kunnen alle componenten in een geïsoleerde omgeving werken zonder dat er conflicten ontstaan tussen verschillende softwareversies.

Docker maakt het eenvoudiger om het systeem op te starten en te gebruiken, zowel lokaal als in de cloud.

Virtuele machines en cloudomgeving

In de ontwikkeling en initiële testing fase maakten we gebruik van virtuele machines die lokaal bij iedereen waren geïnstalleerd. Zeker in deze fasen was het een groot voordeel dat deze een gecontroleerde omgeving boden waarin we al onze tools veilig konden draaien. Later zijn we overgegaan naar een cloudomgeving.

Tijdens de onderzoeksfase werd eerst gekeken naar het gebruik van Oracle Cloud in combinatie met KVM virtualisatie. De beperkingen van onze studentenaccounts hinderde ons echter om deze omgeving correct te kunnen valideren.

Daarom hebben we uiteindelijk gekozen voor Google Cloud. Hier zijn we erin geslaagd de nodige virtuele machines succesvol op te zetten. In deze gesimuleerde productieomgeving konden we een realistischer systeem opbouwen en de verschillende onderdelen van de pipeline beter testen.

Implementatie

Overzicht van de implementatie

In dit project hebben we een proof of concept ontwikkeld, dit om aan te tonen dat het systeem in staat is om stemmen te verwerken en een correcte ranking te bepalen.

We hebben de implementatie opgebouwd als een pipeline bestaande uit verschillende componenten die samenwerken. Elke stap heeft een specifieke taak:

- binnenhalen van data
- verwerken van data
- opslaan van de resultaten

Het doel is niet om een volledig werkend productiesysteem te bouwen, maar om te bewijzen dat de gekozen architectuur en technologieën correct functioneren

Pipeline ontwerp

De pipeline vormt de kern van het systeem. Hij beschrijft hoe de data door de verschillende componenten stroomt. Eerst worden de stemmen gegenereerd, dan wordt de data via Kafka

doorgestuurd naar Spark voor verwerking. Daarna worden de resultaten opgeslagen in Cassandra.

De verwerking kan dankzij deze structuur continu gebeuren en het systeem blijft schaalbaar. De verschillende onderdelen kunnen onafhankelijk van elkaar werken en eenvoudig uitgebreid worden, dit allemaal dankzij de pipeline.

Data ingestie via Kafka

De simulatie van inkomende stemmen werd gedaan aan de hand van een Python-script. Dit genereert continu nieuwe data. Het bootst het stemgedrag van de gebruikers na en zorgt voor een continue datastroom.

Deze gegenereerde stemmen worden doorgestuurd naar Apache Kafka. Kafka is de buffer tussen de input en de verwerking die ervoor zorgt dat zelfs tijdens grote pieken alle stemmen worden opgevangen zonder data te verliezen.

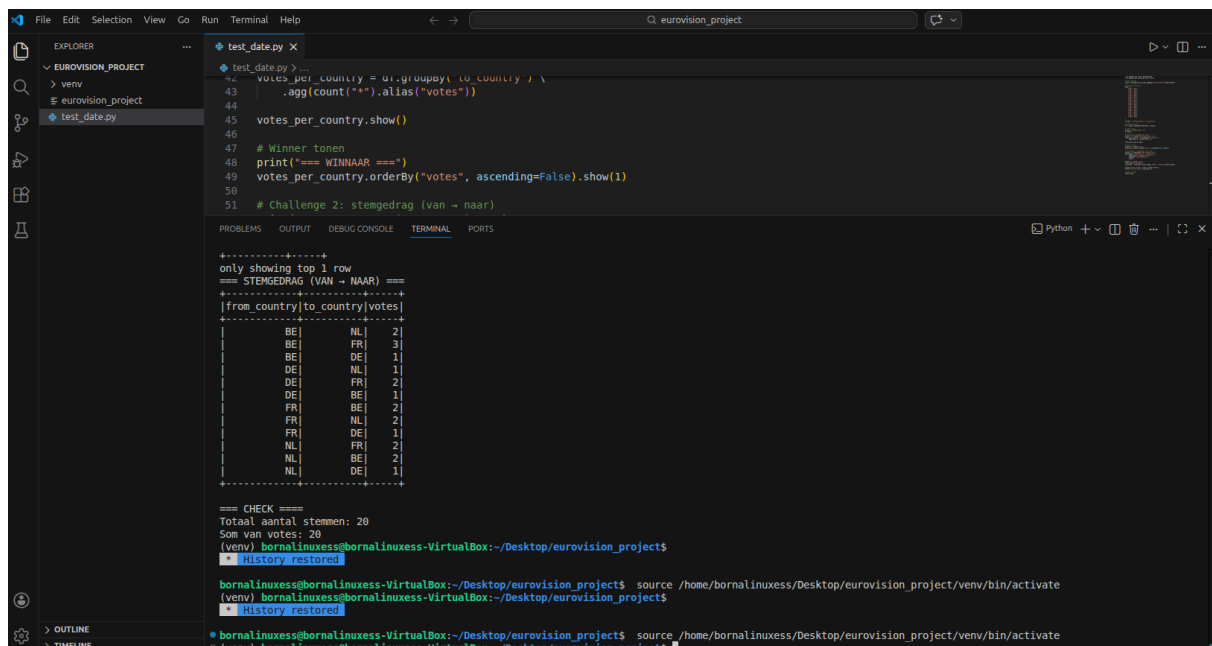
Kafka maakt het mogelijk om de data op een gestructureerde manier door te sturen naar Apache Spark.

Spark verwerking

De verwerking van de data gebeurt met Apache Spark. De data zal worden uitgelezen uit Kafka en in realtime verwerkt.

De stemmen zullen gesorteerd worden per land vooraleer ze worden opgeteld. Op basis van deze gegevens wordt de ranking weergegeven die toont welke landen de meeste stemmen hebben ontvangen.

Een groot voordeel van Spark is de mogelijkheid om data parallel te verwerken. Aangezien dit ervoor zorgt dat er grote hoeveelheden data binnen een beperkte tijd kunnen worden verwerkt.



The screenshot shows a VS Code editor with a file named `test_date.py` open. The script contains the following code:

```
42 votes_per_country = df.groupby('to_country').\
43   .agg(count(**).alias("votes"))
44
45 votes_per_country.show()
46
47 # Winner tonen
48 print("=== WINNAAR ===")
49 votes_per_country.orderBy("votes", ascending=False).show(1)
50
51 # Challenge 2: stemgedrag (van -> naar)
```

The terminal output shows the results of the script execution:

```
only showing top 1 row
=== STENGEDRAG (VAN -> NAAR) ===
+-----+
|from_country|to_country|votes|
+-----+
| BE | NL | 2 |
| BE | FR | 3 |
| BE | DE | 1 |
| DE | NL | 1 |
| DE | FR | 2 |
| DE | BE | 1 |
| FR | BE | 2 |
| FR | NL | 2 |
| FR | DE | 1 |
| NL | FR | 2 |
| NL | BE | 2 |
| NL | DE | 1 |
+-----+
```

Below the table, the terminal shows a check of the total number of votes:

```
=== CHECK ===
Totaal aantal stemmen: 20
Som van votes: 20
(bornalinuxess@bornalinuxess-VirtualBox:~/Desktop/eurovision_project$
* History restored
```

The terminal also shows the command to activate the virtual environment:

```
bornalinuxess@bornalinuxess-VirtualBox:~/Desktop/eurovision_project$ source /home/bornalinuxess/Desktop/eurovision_project/venv/bin/activate
(bornalinuxess@bornalinuxess-VirtualBox:~/Desktop/eurovision_project$
* History restored
```

Figuur 2: Output van het verwerkingsscript waarbij stemgedrag wordt geanalyseerd en een winnaar wordt bepaald

Cassandra opslag

De resultaten worden na de verwerking opgeslagen in Apache Cassandra. Deze database kan grote hoeveelheden data wegschrijven in een korte tijd.

Elke stem wordt opgeslagen als een record met verschillende gegevens zoals een unieke identifier, het land waarop gestemd is, het tijdstip en het IP-adres van de stemmer.

Multi-datacenter setup (Google Cloud)

Om de samenwerking te vergemakkelijken, alsook het systeem realistischer te maken hebben we de pipeline getest in een cloudomgeving met meerdere virtuele machines.

Initieel leed ons onderzoek ons naar het gebruik van Oracle Cloud in combinatie met KVM virtualisatie. We slaagden er echter niet in om deze omgeving correct te valideren met onze studentenaccounts.

Daarom zijn we overgestapt op Google Cloud, hierin konden we meerdere virtuele machines opzetten. Hier konden we de verschillende componenten van het systeem gescheiden draaien, wat beter aansluit bij een gedistribueerde setup.

Deze opstelling bewijst de geschiktheid van het systeem om verder te worden uitgebreid. Wat bewijst dat het systeem geschikt is voor een grotere infrastructuur.

```

saartje_cordier2@cloudshell:~ (project-cl15907ce-0bab-4a45-b70)$ gcloud compute instances list
NAME: dc1-cassandra
ZONE: europe-west1-b
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
INTERNAL_IP: 10.132.0.7
EXTERNAL_IP: 34.156.46.119
STATUS: RUNNING

NAME: dc1-kafka
ZONE: europe-west1-b
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
INTERNAL_IP: 10.132.0.5
EXTERNAL_IP: 34.156.151.21
STATUS: RUNNING

NAME: dc1-spark
ZONE: europe-west1-b
MACHINE_TYPE: e2-standard-2
PREEMPTIBLE:
INTERNAL_IP: 10.132.0.6
EXTERNAL_IP: 34.156.101.238
STATUS: RUNNING

NAME: dc2-cassandra
ZONE: europe-west1-b
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
INTERNAL_IP: 10.132.0.8
EXTERNAL_IP:
STATUS: RUNNING

NAME: dc2-kafka
ZONE: europe-west1-b
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
INTERNAL_IP: 10.132.0.9
EXTERNAL_IP:
STATUS: RUNNING

NAME: dc2-spark
ZONE: europe-west1-b
MACHINE_TYPE: e2-standard-2
PREEMPTIBLE:
INTERNAL_IP: 10.132.0.10
EXTERNAL_IP:
STATUS: RUNNING

NAME: dc3-cassandra
ZONE: europe-west1-b
MACHINE_TYPE: e2-medium

```

Figuur 3: Overzicht van virtuele machines in Google Cloud, verdeeld over meerdere componenten en datacenters

Beperkingen

Het is belangrijk om te begrijpen dat dit een proof of concept is, wat een aantal beperkingen met zich meebrengt.

De data die verwerkt wordt is door ons gesimuleerd en dus niet afkomstig van een echte bron. Hoewel we het systeem grondig getest hebben, was dit op kleine schaal en dus niet met een realistisch aantal stemmen voor tijdens de echte wedstrijd.

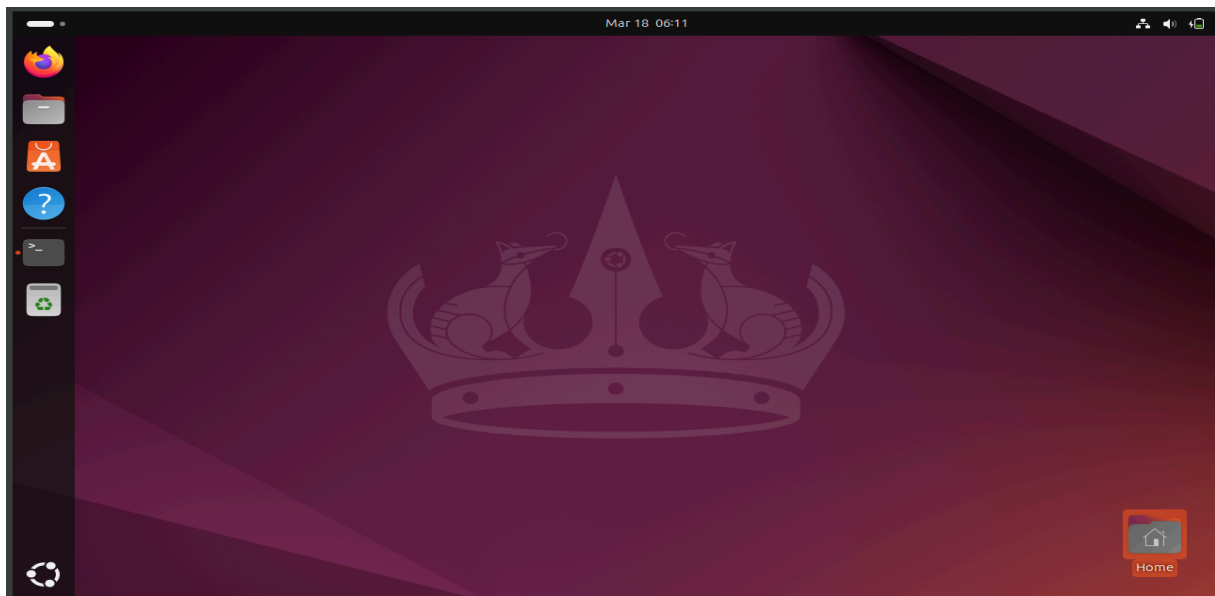
Sommige controles, zoals de validatie van stemmen, zijn slechts beperkt in ons systeem verwerkt. Dit zijn geen grote, abnormale beperkingen voor een PoC. In de toekomst kunnen deze beperkingen altijd worden uitgebreid.

Realisatie & Technische uitwerking

Opzetten van de virtuele machine

Tijdens de ontwikkelingsfase hebben we een virtuele machine opgezet met het besturingssysteem Ubuntu. We lieten deze lokaal draaien om zo een geïsoleerde en gecontroleerde omgeving te creëren.

Dankzij deze omgeving was het mogelijk om alle nodige tools te installeren zonder invloed te hebben op het hostsysteem. Daardoor is het veilig om te experimenteren en fouten op te lossen.



Figuur 4: Virtuele machine met Ubuntu waarin de ontwikkelomgeving voor het project werd opgezet

Installatie en configuratie van Docker

Nadat we de virtuele omgeving hadden opgezet, hebben we Docker geïnstalleerd. Dit om de verschillende componenten in het systeem te laten draaien.

Door het gebruik van containers was het mogelijk om Kafka, Cassandra en Spark onafhankelijk van elkaar op te starten. Dit heeft als gevolg dat de configuratie eenvoudiger werd en alle onderdelen correct konden samenwerken.

De correcte werking van Docker werd door ons na de installatie gecontroleerd door de actieve containers te bekijken.

```
saartje_cordier2@kafka-server:~/eurovision$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ebc58feb9b08	grafana/grafana:latest	"/run.sh"	7 days ago	Up 6 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	eurovision_grafana_1
8363652a1e0c	confluentinc/cp-kafka:7.5.0	"/etc/confluent/docker..."	7 days ago	Up 6 seconds	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp	eurovision_kafka_1
23aff80c0d87	cassandra:4.1	"docker-entrypoint.s..."	7 days ago	Up 7 seconds	7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp	eurovision_cassandra_1
c4fd9f2940b8	confluentinc/cp-zookeeper:7.5.0	"/etc/confluent/docker..."	7 days ago	Up 7 seconds	2181/tcp, 2888/tcp, 3888/tcp	eurovision_zookeeper_1
4655f368c333	bde2020/spark-master:3.2.1-hadoop3.2	"/bin/bash /master.sh"	7 days ago	Up 7 seconds	6066/tcp, 7077/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	eurovision_spark_1

Figuur 5: Overzicht van draaiende Docker containers voor de verschillende componenten zoals Kafka, Spark en Cassandra

Opzetten van Kafka, Spark en Cassandra

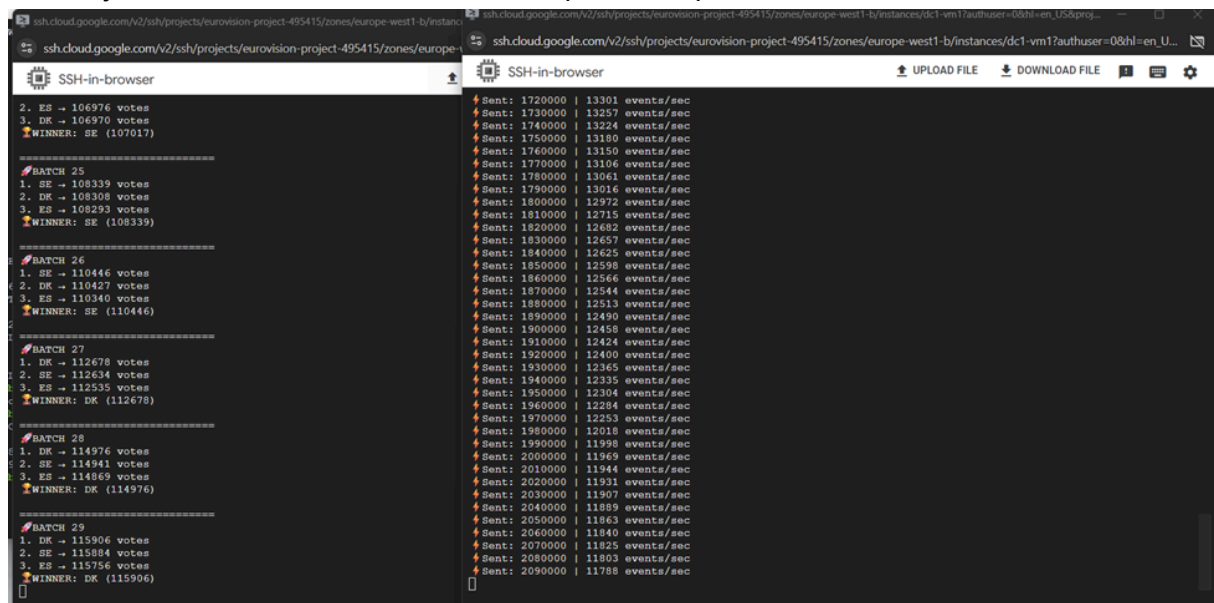
Dan werd het tijd om de belangrijkste componenten van de pipeline op te zetten. Eerst Apache Kafka, dit werd geconfigureerd om de inkomende stemmen te kunnen ontvangen. Vervolgens Apache Spark, voor de verwerking van de data. Als laatste Apache Cassandra, Dit werd geïnstalleerd als database om de resultaten te kunnen opslaan.

Hier was het belangrijk om een goede verbinding op te zetten tussen de verschillende componenten. De data moest namelijk vlot kunnen doorstromen.

Uitvoeren van de pipeline

Na de configuratie werd de pipeline getest door het systeem effectief uit te voeren. Door middel van een Python-script werden stemmen gesimuleerd en doorgestuurd naar Kafka. Het uitlezen van deze stemmen gebeurde dan door Spark, het verwerkte de real-time data tot een ranking. Als laatste werden de resultaten opgeslagen in Cassandra.

Dit toont de correcte samenwerking van onze verschillende onderdelen in ons systeem aan en bewijst dat het in staat is om data van input tot output te verwerken.



Figuur 6: Uitvoering van de data pipeline waarbij grote hoeveelheden stemdata verwerkt worden en de prestatie wordt weergegeven in gebeurtenissen per seconde

Problemen en oplossingen

Tijdens de uitwerking van ons project kwamen er een aantal problemen naar voren.

Belangrijk was het probleem met het opzetten van een cloudomgeving. Tijdens de onderzoeksfase probeerde we nog om gebruik te maken van Oracle Cloud in combinatie met KVM virtualisatie. De correcte validatie is jammer genoeg nooit geslaagd door beperkingen met onze studentenaccounts.

Dit hebben we opgelost door te kiezen voor Google Cloud. Hier konden we meerdere virtuele machines opzetten wat bijdroeg aan een realistischere omgeving.

Er zijn ook verschillende kleine technische uitdagingen geweest. Voorbeelden hiervan zijn de correcte configuratie tussen Kafka en Spark en het debuggen van fouten tijdens de verwerking. Maar door onze iteratieve werkwijze konden we deze fouten oplossen en het hele proces stap voor stap verbeteren.

Testing & Validatie

Testaanpak

Uiteraard moest de werking van het volledige systeem gevalideerd worden, dit deden we aan de hand van een hele hoop gerichte testen. Zo konden we nagaan of de verschillende componenten goed samenwerkten en of de verwerking van de stemmen betrouwbaar verloopt.

Door middel van gesimuleerde data konden testen worden uitgevoerd over de volledige pipeline. Hiermee konden verschillende scenario's worden nagebootst. Zo konden we de correctheid van de verwerking testen, alsook de stabiliteit en performantie bij grote hoeveelheden data.

Uitgevoerde testen

Functionele testen:

- verwerken van inkomende stemmen
- correct doorsturen van data via Kafka
- berekenen van de juiste ranking in Spark
- opslaan van de resultaten in Cassandra

Performantietesten:

Dit werd getest door een hoge stroom aan data te genereren, zo konden we controleren of het systeem stabiel bleef werken.

Betrouwbaarheid en continuïteit:

Het was ook nodig om na te gaan of het systeem blijft functioneren wanneer er continu nieuwe stemmen blijven binnenkomen.

Analyse van resultaten

Uit de testen trekken wij de conclusie dat de pipeline correct functioneert. De verwerking van de stemmen en omzetting naar de ranking verloopt correct. Het systeem lijkt schaalbaar te zijn om grote hoeveelheden data te kunnen verwerken zonder veel vertraging. Deze resultaten zijn zichtbaar in de uitvoer van de pipeline zoals weergegeven in het vorige hoofdstuk.

Hoewel de testen werden uitgevoerd in een gesimuleerde omgeving zijn wij van mening dat de resultaten een duidelijk beeld geven over het potentieel om dit systeem in een realistische situatie in te zetten. Het is duidelijk dat de gekozen technologieën en architectuur geschikt zijn voor de verwerking van deze grote datastromen.

Resultaten

We zijn erin geslaagd een werkend proof of concept te ontwikkelen dat in staat is stemdata te verwerken via een pipeline. Het systeem ontvangt inkomende stemmen en buffert deze in Kafka tot Spark ze kan verwerken. Er wordt dan een correcte ranking opgesteld op basis van het aantal stemmen per land. Tot slot worden de resultaten opgeslagen in Cassandra. De pipeline is in staat om continu nieuwe inkomende data te blijven verwerken.

De testen bewijzen dat het systeem grote hoeveelheden data in een korte tijd kan verwerken. Dit dankzij het gebruik van Apache Kafka en Spark die ervoor zorgen dat meerdere events per seconde kunnen worden verwerkt.

De opstelling van de verschillende virtuele machines in een cloudomgeving toont aan hoe schaalbaar het systeem is. De capaciteit van het systeem kan makkelijk verhoogd worden door resources toe te voegen.

Conclusie

In dit project werd een proof of concept ontwikkeld voor een systeem dat stemdata van de Eurovision Song Contest kan verwerken via een data pipeline. Door gebruik te maken van Apache Kafka, Apache Spark en Apache Cassandra werd een werkende oplossing gerealiseerd die stemmen kan verwerken en omzetten naar een correcte ranking. Daarnaast werd het systeem getest in een gedistribueerde cloudomgeving, wat de schaalbaarheid en flexibiliteit aantoont.

Tijdens het project werd duidelijk dat het verwerken van grote datastromen een doordachte architectuur vereist. Het gebruik van een pipeline en gespecialiseerde technologieën maakt het mogelijk om deze uitdaging efficiënt aan te pakken. Ook containerisatie en cloudoplossingen bleken essentieel om een flexibele en schaalbare omgeving op te zetten.

Hoewel het proof of concept succesvol is, zijn er nog uitbreidingsmogelijkheden. In de toekomst kan het systeem verbeterd worden door:

- gebruik van echte data
- meer uitgebreide validatie
- verdere optimalisatie voor grotere schaal

Daarnaast kan Hadoop verder onderzocht worden voor historische data-analyse.

Op basis van dit project kan geconcludeerd worden dat een data pipeline een geschikte oplossing is voor het verwerken van grote hoeveelheden stemdata.

De gekozen technologieën vormen een sterke basis voor een mogelijke verdere uitwerking naar een volwaardig systeem.

Referenties